

EduTech Quantum Learning Platform

Project Plan & Task Allocation

A narrative-driven quantum computing course experience, planned and built end to end by a single engineer — with effort estimates, checkpoints, and definitions of done.

Prepared for Dr. Hammad Tanveer Butt, PI — Quantum Computing Lab, NUST

Prepared by Taha Mehdi — Software Engineer

Date July 2026

Contents

1. Project Brief.....	3
2. What Was Studied on Brilliant.org.....	3
3. The Product: Translating This Into the EduTech App.....	4
4. System Architecture.....	4
5. Design & Planning — Status.....	4
6. Working Model — Solo Ownership.....	5
7. Scope Priorities (MoSCoW).....	6
8. Phase Plan.....	7
9. Timeline, Milestones & Checkpoints.....	10
10. Risks & Mitigations.....	10
11. Deliverables Checklist.....	11

Effort key used throughout Section 8: **S** ≈ 0.5 day **M** ≈ 1 day **L** ≈ 2 days **XL** ≈ 4 days. Estimates assume solo, focused effort against the already-finalized specs.

1. Project Brief

This document is the project plan for Task 2 of the internship: designing a narrative-driven course experience for three quantum computing topics, and building a working web application in the same instructional style as Brilliant.org — planned and delivered end to end by a single engineer, covering content design, backend, frontend, security, and deployment.

The scope, in the supervisor's own task wording, is:

- Task 2a — Design a narrative/storyline and three courses on: Quantum Computing Hardware, Quantum Algorithms, and Quantum Machine Learning.
- Task 2b — In the same manner, develop the front end and back end of an app for EduTech.

Task 1 (enrolling in Brilliant's own courses and screenshotting each screen before/after attempting questions) is treated as the working UX specification for this build — every screen pattern designed should be traceable back to something observed on the live platform.

2. What Was Studied on Brilliant.org

Before planning the build, Brilliant's course catalog, course pages, help center, and public product write-ups were reviewed to extract the mechanics worth reproducing.

2.1 Content Structure

Brilliant organizes content as: Learning Path → Course → Chapter → Lesson → Screen, with a Practice Set attached to each lesson. Lessons mix short explanations with an interactive question on the same screen; practice sets remove the visual hints and test independent recall.

2.2 Interaction Style

Lessons are visual and hands-on rather than video-based — learners manipulate a diagram, drag a component, or answer an embedded question and get immediate right/wrong feedback before moving to the next screen. This is the exact before/after pattern Task 1 captured.

2.3 AI Tutor Layer (Koji)

Brilliant's newer tutor, Koji, sits inside the lesson and can see what the learner has done so far, then guides step-by-step using Socratic questions rather than giving the answer outright. This is noted as a strong differentiator and a possible later-phase stretch goal — explicitly out of scope for this build (see Section 7).

2.4 Gamification & Progression

Motivation is driven by a small set of mechanics: XP earned per completed lesson/question, daily Streaks, weekly League rankings against other learners, and a Level system per topic that gives a simplified, motivating sense of progress.

2.5 Access Model

Free accounts move through courses sequentially with daily limits; a paid tier removes limits and allows jumping to any lesson. For this academic build, all content stays open, but the lesson-locking logic is worth designing for, in case a staged release is wanted later.

3. The Product: Translating This Into the EduTech App

This model is reproduced for three courses — Quantum Computing Hardware, Quantum Algorithms, and Quantum Machine Learning — each broken into chapters and lessons with embedded interactive questions, followed by a practice set, and tracked with XP/streaks so progress feels the same as on Brilliant.

- Course catalog → 3 quantum courses, each with a short narrative framing (a storyline that motivates why the topic matters before diving into mechanics).
- Lesson player → sequential screens mixing explanation + one interactive question, with immediate feedback.
- Practice sets → unscaffolded checks after each lesson.
- Progress system → XP, streaks, and a simple per-course level indicator.
- Admin/content tool → so new lessons can be added without a code deploy.

Content-accuracy note: the courses are grounded directly in Dr. Hammad's own workshop sequence — Day 3's physical-implementation content for Quantum Computing Hardware, Day 4–5's Grover's/Shor's/RSA sequence for Quantum Algorithms, and Day 2's Hilbert-space framing as the bridge into Quantum Machine Learning. This is worth stating explicitly in the final presentation: it is an interactive version of his own material, not a separately invented curriculum.

4. System Architecture

- Frontend: React (web-first, responsive), component library built around the lesson-screen pattern.
- Backend: Node.js + Express REST API, strict Controller → Service → Repository layering with no exceptions.
- Database: PostgreSQL — fits the Course → Chapter → Lesson → Screen → Question hierarchy.
- Auth: JWT access tokens with rotating single-use refresh tokens, argon2id password hashing, httpOnly cookies.
- Hosting (demo stage): backend on Render/Railway, frontend on Vercel/Netlify, managed Postgres instance.
- **Known constraint:** free-tier hosting on Render/Railway commonly cold-starts or throttles after inactivity, and free Postgres instances have storage/connection limits. Flagged here so it is planned around — e.g. a warm-up request before a live demo, or a temporary paid tier for demo day — rather than discovered live in front of Dr. Hammad.

5. Design & Planning — Status

A full pre-implementation design pass has been completed and finalized before any application code was written, so the build phases that follow execute against a stable, internally consistent spec rather than evolving requirements.

Deliverable	What it contains	Status
Data Model	Full ER diagram and column-level spec for User, Cohort, CohortEnrollment, Course, Chapter, Lesson, Screen, Question, PracticeSet and junction tables — the full Course→Chapter→Lesson→Screen→Question hierarchy plus attempts, progress, and XP/streaks.	Finalized
API Contract	Endpoint-by-endpoint spec: auth & users, content CRUD, practice sets & attempts, gamification, admin — including authorization patterns (role checks, ownership checks, server-derived fields, role-based field shaping) and a standardized error shape.	Finalized
Security Architecture	The concrete mechanisms behind every policy in the API contract: argon2id password hashing (OWASP-current parameters), JWT lifetimes and rotation, RBAC middleware, rate limiting, input validation, CORS/transport security, and audit logging.	Finalized
Application Architecture	The Controller → Service → Repository layering rule with no exceptions, backend folder structure, naming conventions, middleware/validator organization, and a frontend folder structure that deliberately mirrors the backend.	Finalized
Threat Model	A STRIDE-style pass across three actors (unauthenticated outsider, learner, instructor), used to drive the mitigations captured in the security architecture and audit-log design.	Finalized
Course Narratives (Spine)	Chapter-level storyline for all 3 courses — core question, narrative throughline, and a 6-chapter table with motivating hooks for each. This is Task 2a at the chapter level.	Finalized
Full Course Content ×3	Lesson- and screen-level breakdown for Quantum Machine Learning, Quantum Algorithms, and Quantum Computing Hardware — every chapter expanded into individual lessons with screen-by-screen content.	Finalized

Change control: if implementation surfaces a needed change to any finalized spec (a missing column, a route that doesn't quite fit), the change is logged as a short amendment note appended to that spec document — the same convention the specs already use internally — rather than silently drifting from what's written or rewriting this plan each time.

6. Working Model — Solo Ownership

This is a one-person build: content design, backend, frontend, security, and deployment all sit under a single owner. That has one direct planning consequence worth stating up front — work that would otherwise run in parallel across two people instead runs sequentially, so the phase plan below is ordered by what actually blocks a demo-able build, not by role.

Area	Approach
Content design (Task 2a)	Fully authored before implementation begins, so the app is built against finished lessons rather than placeholder copy.
Data model, API, security, application	Fully specified before implementation begins, so backend and frontend can each

Area	Approach
architecture	be built once, correctly, rather than reworked as the shape of the system changes underneath them.
Backend + content pipeline	Built first — the frontend has nothing real to call against until the API and a way to load real content both exist.
Frontend + shared widgets	Built second, directly against the finalized API contract, to avoid churn from an unstable backend shape.
Security hardening	Built into the backend implementation phase rather than treated as a separate later pass — cheaper to build correctly the first time than to retrofit.
Integration, QA, deployment, demo prep	The final phase, once both layers exist and can be tested end to end.

7. Scope Priorities (MoSCoW)

Stated up front so a time-crunch decision later is a lookup, not a debate. If the timeline in Section 9 slips, cuts happen bottom-up: Won't Have is already excluded; Could Have items go first; Should Have items degrade gracefully rather than get cut outright (e.g., two courses fully wired in, the third left as finalized content ready to load but not yet demoed) — consistent with the adoption strategy's guidance to favor depth over breadth for this evaluator.

MUST HAVE

- One fully working course end-to-end: real content, lesson player, embedded questions, practice sets.
- Auth (signup/login/logout/refresh) with the security mechanisms specified in Section 5.
- Core progress tracking: attempts recorded, XP awarded.
- A secure, stable, deployed staging build reachable by a link.

SHOULD HAVE

- All three courses populated via the content pipeline (Section 8, Phase 2).
- Streaks and a simple per-course level indicator.
- Admin endpoints for content entry.
- Full responsive layout and an accessibility pass.

COULD HAVE

- League rankings (competitive comparison across learners).
- Transitions/animations between lesson screens.
- Staged lesson-locking logic — only needed if Dr. Hammad wants a phased rollout to a cohort.

WON'T HAVE (this round)

- A Koji-style AI tutor layer — logged as a future differentiator, not this build.
- A full drag-and-drop circuit-builder simulation beyond the shared MVP widget set.
- Multi-cohort analytics dashboards beyond the basic admin views.

8. Phase Plan

Phase 0 — Discovery

COMPLETE

- Brilliant.org screen-pattern research and MVP feature list.

Definition of Done — A screen-pattern reference doc and MVP feature list exist and are used as a checklist during the build.

Phase 1 — Planning & Design

COMPLETE

- Data model, API contract, security architecture, application architecture, and threat model — all finalized (Section 5).
- Course narratives and full lesson/screen-level content for all 3 courses — finalized (Section 5).
- Wireframes and a written design system are intentionally folded into Phase 3 rather than kept as a separate step — building directly from the finalized, screen-level course content is faster than a separate wireframe pass first.

Definition of Done — All six design docs are finalized, internally consistent, and cross-referenced; course content is complete to screen level for all three courses.

Phase 2 — Backend Build & Content Pipeline

Note on scope: the three course documents are ~60–70k characters of narrative lesson content each. Turning that into rows in the Screen/Question/PracticeSet tables is real engineering, not a data-entry afterthought — it needs a parser/mapper and a validation pass, sized honestly below rather than folded silently into "deployment."

Task	Output	Effort
Repo/DB setup, migrations, seed script for one demo course	Working DB matching the finalized schema	M
CI pipeline: lint + test on every push	Green/red build status before code is trusted	S
Auth endpoints: signup, login, refresh, logout	Auth API	M
Course/Chapter/Lesson/Screen CRUD endpoints	Content API	L
Content migration pipeline: parse course markdown into Screen/Question/PracticeSet rows against the finalized schema, with a validation pass	Reusable content loader — used again in Phase 6 for the flagship course	L
Practice set + attempt-recording endpoints	Progress API	M
XP / Streak / League calculation logic	Gamification API	M
Admin endpoints for content entry	Admin API	M
Security mechanisms from the architecture doc: argon2id, JWT rotation, RBAC middleware, rate limiting, input validation, audit log	Hardened-by-default API	L
Unit tests + env/secrets configuration	Test suite	M

Definition of Done — Every endpoint in the API contract responds correctly against the finalized schema; every mechanism in the security architecture doc is implemented and testable; the content pipeline can ingest a course markdown file into the DB without manual SQL; CI is green.

Phase 3 — Frontend Build & Shared Widgets

Note on scope: the course spine names six shared interactive widgets reused across all three courses. The Bloch sphere in particular — a manipulable 3D state-vector view, per the build-strategy goal of "simulations, not animations" — is the single hardest piece of frontend work in the project and is sized accordingly rather than folded into a generic "components" line.

Widget	Notes	Effort
Bloch sphere (interactive 3D)	Apply X/H/S/T gates, watch the state vector move in real time — the flagship simulation.	XL
Topology diagram	Read/interact with a QPU connectivity graph.	M
Drag-to-order	Used for protocol/sequence questions (e.g. teleportation steps).	M
Amplitude bar chart	Visualizes measurement probabilities.	S
MCQ	Standard embedded question type.	S
Numeric input	Standard embedded question type.	S
Task	Output	Effort
Project scaffold, routing, base layout (folder structure per application architecture doc)	App shell	S
Sketch the lesson question state machine (unattempted → correct/incorrect → retry) before building the renderer — reused on every screen	Question interaction spec	S
Reusable components: card, button, progress bar, modal, question renderer, XP/streak badge	Component library	M
Build screens: landing, auth, catalog, lesson player, practice set, dashboard	Core screens	L
API integration & state management against the finalized contract	Connected frontend	M
Before/after answer-state UX (mirrors Task 1 screenshots)	Question interaction flow	M
XSS-safe content rendering; secure token handling (httpOnly cookies, not localStorage); CSRF mitigation	Secure frontend	M
Responsive layout (mobile + desktop)	Responsive build	M

Definition of Done — All six shared widgets work standalone against mock data; all core screens are built and wired to the live API; one full lesson can be completed end-to-end on both desktop and mobile.

Phase 4 — Integration & QA

Task	Output	Effort
Connect frontend to live backend (remove mocked data)	Integrated build	M
End-to-end test: signup → enroll → lesson → practice set → XP/streak update	Test report	M
Dependency vulnerability scan; manually verify each threat-model mitigation against the running app, not just the design doc	Security sign-off	M
Basic load test (e.g. k6/Artillery) simulating ~50 concurrent users through the core flow	Load test report	M
Bug-fixing pass	Stable build	L

Definition of Done — The signup → enroll → lesson → practice set → XP/streak flow passes end-to-end with no mocked data; the dependency scan is clean or exceptions are documented; every threat-model mitigation has been manually verified against the running app; the app holds up under the load test.

Phase 5 — Polish

Task	Output	Effort
Transitions/animations between lesson screens	Polished UX	M
Loading, empty, and error states	Edge-case UI	M
Accessibility pass (contrast, keyboard nav)	Accessibility checklist	M
Cross-device/browser visual QA	QA sign-off	M
API performance pass + error tracking/monitoring (e.g. Sentry free tier)	Stable, observable API	M
Informal accuracy review of the simulations with a workshop PhD student (e.g. Dr. Madiha Khalid, Dr. Abdul Razzaque) — scheduled, not left as a hope	Reviewed-by credit; corrections logged and applied	S

Definition of Done — No unhandled loading/empty/error state remains in the core flow; the accessibility checklist passes; the load test from Phase 4 still passes after polish changes; the accuracy review has happened and any corrections are applied.

Phase 6 — Deployment & Demo Prep

Task	Output	Effort
Deploy backend and frontend to staging; account for the hosting constraint noted in Section 4	Live demo link	M
Run the content pipeline (Phase 2) to fully populate the flagship course — Quantum Algorithms, for its clean narrative arc to Shor's/RSA	Demo course	M
Prepare presentation covering Task 1 findings, the finalized design docs, and a live demo	Presentation for Dr. Hammad	M

Definition of Done — Staging links are live and stable; the flagship course is fully populated via the content pipeline and playable start to finish; the presentation deck covers Task 1 findings, the design docs, and a live demo.

9. Timeline, Milestones & Checkpoints

Weeks below are derived from the effort estimates in Section 8 (summed per phase, ~5 focused days per week), not assigned arbitrarily. Three checkpoints with Dr. Hammad are built in deliberately, so direction is corrected early rather than discovered at the final demo.

Week	Phase	Milestone
Weeks 1–2	Phase 0–1	Discovery and full design pass complete: data model, API contract, security architecture, application architecture, threat model, and all 3 courses to lesson/screen level. (Complete.)
Weeks 3–5	Phase 2	Backend and content pipeline complete and self-consistent with the finalized contract. — Checkpoint 1 with Dr. Hammad: walk through the API, DB, and content-pipeline output; sanity-check the content-to-schema mapping before frontend work depends on it.
Weeks 6–8	Phase 3	Frontend and all six shared widgets built against the live API; one full lesson playable end-to-end. — Checkpoint 2 with Dr. Hammad: live demo of one working lesson, including the Bloch sphere widget, for early feedback.
Week 9	Phase 4	Integrated build; end-to-end flow tested; security sign-off against the threat model; load test passed.
Week 10	Phase 5	Polish pass complete; accuracy review completed and corrections applied.
Week 11	Phase 6	Staging deployment, flagship course fully populated, final presentation and live demo to Dr. Hammad. — Checkpoint 3: final review.

Total: ~11 weeks (≈2.5 months) from today, comfortably inside the 3-month adoption-strategy window, with roughly two weeks of slack unaccounted for as buffer.

If the timeline still needs to compress: cut scope per Section 7 (Could Have, then Should Have) before cutting the checkpoints or the Phase 4/5 QA tasks — a late, unreviewed feature is a worse outcome than a smaller, verified one for this specific evaluator.

10. Risks & Mitigations

Risk	Mitigation	Status
No second reviewer for code or content before it ships	Implementation is checked against the finalized data model, API contract, and security architecture docs directly, not against memory.	Ongoing
Physics accuracy has no automatic second pair of eyes	Cross-check simulations against Nielsen & Chuang / Qiskit conventions; informal accuracy review scheduled in Phase 5, Week 10 (Section 8).	Scheduled — Week 10
A sequential build (no parallelism) puts pressure on	Backend-first, then frontend, avoids the larger risk of building UI against an API shape that later changes;	Planned

Risk	Mitigation	Status
the timeline	the MoSCoW cut-line in Section 7 is the fallback.	
Security hardening deferred to "later" historically slips in solo projects	Folded directly into Phase 2 rather than left as a separate later phase; re-verified against the running app in Phase 4.	Addressed by re-sequencing
Content migration from markdown to DB introduces silent data-mapping errors	Dedicated content-pipeline task with a validation pass (Phase 2); spot-checked again when the flagship course is populated in Phase 6.	Planned
Free-tier hosting cold-starts or throttles during the live demo	Known constraint documented in Section 4; warm-up request or temporary paid tier planned for demo day (Phase 6).	Planned
The app has never been tested under realistic concurrent load	Load test task added to Phase 4 (~50 concurrent users through the core flow) before polish begins.	Planned

11. Deliverables Checklist

- Task 1 screenshots (Brilliant.org, before/after each screen)
- Storyline outline for the 3 quantum courses — complete
- Full lesson/screen-level content for all 3 courses — complete
- Data model, API contract, security architecture, application architecture, threat model — complete
- Content migration pipeline (markdown → DB) — Phase 2
- Six shared interactive widgets, including the Bloch sphere — Phase 3
- Working, integrated frontend + backend app (staging link) — Phases 2–4
- Security sign-off against the threat model, including manual verification — Phase 4
- Load test report (~50 concurrent users) — Phase 4
- Accuracy review credit from a workshop PhD student — Phase 5
- Final presentation and live demo for Dr. Hammad — Phase 6

End of document.